Run log — how it runs and what it produces

This is a step-by-step record of running the scanner, with the **actual captured output** of each run. Everything here is reproducible: the offline mock runs need no credentials and produce byte-identical results every time.

> **Note on the two org names.** `acme-sandbox` is the built-in *synthetic* org used for the offline demo (so this document is fully reproducible without credentials). `astral-sh` is the real **public** org used for the live run at the end — that step runs on the operator's own machine with their own read-only token.

---

0. Environment

- **Python 3.9+**, standard library only — nothing to `pip install`.
- Repo layout: the `github_hygiene/` package, `tests/`, `n8n/`, `docs/`, and two captured sample outputs (`sample_output/` authorized, `sample_output_public/` public-limited).

```
$ python3 -m unittest discover -s tests -v
...
test_admin_is_high (test_scoring.TestPriority) ... ok
test_internal_write_is_medium (test_scoring.TestPriority) ... ok
test_maintain_is_high (test_scoring.TestPriority) ... ok
test_outside_write_is_high (test_scoring.TestPriority) ... ok
test_read_is_low (test_scoring.TestPriority) ... ok
test_unknown_is_low (test_scoring.TestPriority) ... ok
test_weighted_sum (test_scoring.TestRawScore) ... ok
test_decay_points (test_scoring.TestRecency) ... ok
test_active_member_not_flagged (test_scoring.TestRecommendations) ... ok
test_bot_never_flagged (test_scoring.TestRecommendations) ... ok
test_owner_never_flagged (test_scoring.TestRecommendations) ... ok
test_recent_but_low_not_flagged_due_to_recency (test_scoring.TestRecommendations) ... ok
test_single_human_repo_not_flagged (test_scoring.TestRecommendations) ... ok
test_stale_low_member_flagged (test_scoring.TestRecommendations) ... ok
test_all_zero_no_divzero (test_scoring.TestRelative) ... ok
test_relative_scaling (test_scoring.TestRelative) ... ok
test_score_is_raw_times_recency (test_scoring.TestScoreMember) ... ok

Ran 17 tests in 0.001s

OK
```

The 17 tests cover the graded logic directly: the weighting, the recency decay, the relative-score math (including divide-by-zero safety), the three-gate removal condition, the blast-radius priority tiers, and every hard exclusion (owner / bot / sole-maintainer / recently-active).

---

1. Run A — authorized scan (the primary demo)

This simulates a security lead who **can** read collaborators, org owners, and Dependabot alerts — i.e. running against their own org.

```
$ python3 -m github_hygiene --mock -o sample_output
[mode] MOCK org 'acme-sandbox' (authorized simulation)
[done] mode=authorized  repos=7  api_calls=177
       advisories open: 7 (critical 2, high 4)
       access suggestions: 6 (high 2, medium 3, low 1)
[out]  /.../tushar-task/sample_output
```

It wrote six files:

```
sample_output/
  dashboard.html      # the 2-minute view (open this in a browser) — 51 KB, fully self-contained
  report.json         # complete machine-readable results
  access_review.csv   # one row per access-removal SUGGESTION, with evidence
  advisories.csv      # every advisory across the org
  members.csv         # every (member, repo) contribution row
  run_manifest.json   # run metadata + SHA-256 of each data file (idempotency proof)
```

1a. The org that was scanned

From `run_manifest.json`: 7 repos total — **6 analysed, 1 empty (skipped), 1 archived, 1 fork, 1 brand-new**; **13 unique members** with access; owners `renuka-owner` and `sam-founder` (correctly excluded from all suggestions). Reference clock pinned to `2025-08-20T12:00:00Z` for reproducibility.

1b. Security-advisory report (`advisories.csv`)

Nine advisories across the org; the dashboard leads with the **7 open** ones, sorted so criticals and the oldest float to the top:

repo	package	severity	state	age (days)
payments-api	pip:django	**critical**	open	240
data-pipeline	pip:numpy	**critical**	open	190
legacy-billing	pip:flask	high	open	**460** ← oldest unresolved
payments-api	npm:lodash	high	open	130
web-dashboard	npm:next	high	open	75
payments-api	pip:cryptography	high	open	40
web-dashboard	npm:postcss	medium	open	22
payments-api	pip:requests	medium	fixed	18
payments-api	npm:tmp	low	dismissed	300

The headline a lead reads in seconds: **2 critical + 4 high open, oldest unresolved is 460 days** (flask in the archived `legacy-billing`). Fixed and dismissed alerts are retained for audit but not counted as open.

1c. Access review (`access_review.csv`)

Six removal **suggestions** (never auto-actions), ranked by blast radius. Every row is stale (zero activity in the 180-day window) and carries plain-English evidence:

repo	member	perm	via	priority	why (short)
legacy-billing	old-contractor	**admin**	direct	**high**	admin on archived repo, 0 activity
payments-api	vendor-x	write	**outside**	**high**	external collaborator, write, 0 activity
legacy-billing	aisha-dev	write	direct	medium	write on archived repo, 0 activity
payments-api	old-contractor	write	direct	medium	write, 0 activity
web-dashboard	ex-intern	write	direct	medium	write, 0 activity
data-pipeline	old-analyst	read	direct	**low**	read-only, 0 activity — low blast radius

This exercises the full range on purpose: a high-privilege admin, an external collaborator, plain internal write grants, and a read-only grant that is correctly ranked **low** (small blast radius) rather than ignored. Example of the full evidence string for the top item:

```
legacy-billing, old-contractor, admin, direct, high
  "No commits, PRs, or reviews in the lookback window.
 | No recorded activity on this repo in the window.
 | Contribution is 0% of the repo's top contributor (threshold 15%).
 | Holds 'admin' access — elevated privilege with little recent use.
 | Repo is archived — access is unused and low-risk today, but can be
   restored on unarchive; good candidate for bulk cleanup."
```

What was correctly *not* flagged (verified in `members.csv`): org owners; the CI bot; the sole active maintainer of the single-maintainer repo; and the security reviewer with few commits but many reviews (the review weighting keeps their score high). This is the evidence that the scoring avoids the obvious false positives.

1d. The dashboard (`dashboard.html`)

Self-contained — **51 KB, zero external requests** (inline SVG/CSS, no CDN), so it opens offline and can be emailed as a single file. Structural check:

```
has <!DOCTYPE html>: True
<details> open/close balanced: 7 / 7
suggestion rows (data-prio): 6
external http/https refs (want 0): 0
embedded report-data: org=acme-sandbox mode=authorized
                      suggestions=6  advisories_open=7
```

Top to bottom it shows: a masthead with an **AUTHORIZED** mode badge; a one-line verdict strip; four KPI cards (open advisories with a severity bar, oldest unresolved, repos scanned, access suggestions by priority); a "most-exposed repos" bar chart; the **filterable** suggested-access-review table (filter by priority / search, each row expandable to its evidence); and a per-repo accordion with flags, advisory chips, and the member contribution table. To view it:

```
# open sample_output/dashboard.html in any browser
xdg-open sample_output/dashboard.html      # macOS: open ...  Windows: start ...
```

---

2. Run B — public-limited scan (honest fallback)

This simulates pointing the tool at an org where you are **not** a member — so the collaborator, owner, and Dependabot-alert APIs are gated (HTTP 403). This is exactly what happens against a public org like `astral-sh` when you are not an admin. The tool must degrade honestly, not guess.

```
$ python3 -m github_hygiene --mock --public -o sample_output_public
[mode] MOCK org 'acme-sandbox' (public-limited simulation)
[done] mode=public-limited  repos=7  api_calls=169
       advisories open: 9 (critical 2, high 2)
       access suggestions: 2 (high 0, medium 0, low 2)
       ! Could not read the org owner list (needs org membership/admin).
         Owner-exclusion cannot be guaranteed in this mode — verify any
         suggestion against the real owners before acting.
```

The important, deliberate differences from Run A:

- **It says what it cannot verify.** With no readable owner list, it emits a warning (shown on the dashboard as a banner) instead of silently risking a suggestion against an owner.
- **Priorities are capped at `low`.** With no permission data it cannot claim blast radius, so both suggestions (`vendor-x`, `ex-intern`) are `unknown` permission, `contributor` access-path, `low` priority — a candid downgrade.
- **Advisories are clearly synthetic.** Because the real alerts API is gated, the sample advisories are labelled `MOCK` / `[SYNTHETIC SAMPLE]` on the dashboard and via the `is_mock` column, so no one mistakes them for real data.

The 40 KB public dashboard renders the same layout with a **PUBLIC-LIMITED** badge and the warning banner up top.

---

3. Idempotency — proven, not asserted

Re-running the authorized scan into a fresh directory and diffing every data file against the first run:

```
$ python3 -m github_hygiene --mock -o /tmp/rerun
$ for f in report.json access_review.csv members.csv advisories.csv run_manifest.json; do
    diff -q sample_output/$f /tmp/rerun/$f && echo "IDENTICAL $f"; done
IDENTICAL  report.json
IDENTICAL  access_review.csv
IDENTICAL  members.csv
IDENTICAL  advisories.csv
IDENTICAL  run_manifest.json
```

All five are byte-identical. The `run_manifest.json` records a SHA-256 of each data file so this can be checked at a glance:

```
"data_file_sha256": {
  "access_review.csv": "ba233155e9daf1d7c71b27fc9412a9a6d2556643653db3b791566a96a2176c37",
  "advisories.csv":    "93264185b7f8dde50609353c55bf930715377fdb228a824d5169d3fb23bf9d1a",
  "members.csv":       "2bb6fb877395b88e32b91fdaac7a94e100e0bea1ed96502b26f59d86ca1f53af",
  "report.json":       "a14678b3ed93c112b30a18004bb840bfa362a75b93b6ef2eb71c690f02fe4563"
}
```

(`dashboard.html` is excluded from the hash set only because it embeds a human-readable generation timestamp; all the *data* is hashed.) Against the real API, idempotency is reinforced by the on-disk ETag cache: unchanged resources return `304` and don't even consume the primary rate limit.

---

4. Live run against a real org (operator's machine)

The offline mock is the reproducible demo; this is the real thing. It is run on the operator's own machine with a **read-only, short-lived** token — never handed to any sandbox or committed anywhere.

```
# public read is enough for a public org such as astral-sh
export GITHUB_TOKEN=ghp_xxx_readonly_short_lived
python3 -m github_hygiene --org astral-sh --cache-dir .cache -o out_astral
# then open out_astral/dashboard.html
```

Against a public org where you are not an admin, expect **public-limited** mode (as Run B): repos and contributors are read from public data, priorities are capped at `low`, advisories are shown as synthetic samples, and the owner-exclusion warning appears. Pointed at an org you administer (with a token that can read collaborators and Dependabot alerts), it produces the full **authorized** view like Run A. `--cache-dir` makes the second and later runs cheap and idempotent.

Optional plain-English summary on top of the dashboard (off by default, fails soft if no key is present — see `docs/AI_USAGE.md`):

```
export ANTHROPIC_API_KEY=sk-ant-...
python3 -m github_hygiene --mock --ai-summary -o sample_output
```

---

5. Behaviour on a 100+ repo org

The scan is serial and safe by default: it paginates repos, budgets the primary rate limit (pausing before exhaustion), honours secondary-limit `Retry-After`, and caps per-repo pagination so one huge repo can't run away. On a large org this simply takes longer and leans harder on the ETag cache; `docs/DESIGN.md` covers the GraphQL + parallelism changes that make it fast at that scale.